AI Product Sprint — Codex Execution Playbook

VERSION 2.0 **Built from:** Prompt Kit v1, RoamSense PRD v6.1, your master brief • **Written:** 3 Oct 2026

Tags used in this document

- [PUBLIC-DOCS] = found in public documentation for a package or tool with the same name. Not confirmed against your installed copy.
- [NEEDS CONFIRMATION] = I do not have this information. A default assumption is stated.
- [YOUR ACTION] = something only you can do (an account, a dashboard, a real phone).

1. How to Use This Guide

Two ways to use it

- **Reference manual:** read Sections 1-9 once, then use Sections 10-18 as a lookup.
- **Guided mode:** type "Start the sprint" in this Claude chat. I then give you ONE step at a time, in the format in Section 19, using the steps in Section 10 and the prompts in Section 13.

Five rules that run through everything

1. **P0 first.** Working core, then integration, tests, deployment, polish.
2. **One step at a time.** Do not start a step until the previous one passes its "Done when" check.
3. **Trust tests, not claims.** Codex saying "done" means nothing until the test output is shown.
4. **Deploy early.** A blank-but-live skeleton goes up in Phase 1, not Phase 6.
5. **The PRD wins.** If a prompt, this guide or Codex disagrees with PRD v6.1, the PRD wins. Record the conflict in `docs/DECISIONS.md`.

Needs Confirmation register

#	Item	Why it matters	Default I used	Resolved by
NC-1	Exact Product Space deadline, and whether "5 days" and "48 hours" mean elapsed or working time	Three products plus 3 PPTs and 3 videos may not fit (see Section 9)	48 h for RoamSense counts from "Start the sprint"; other products are lean	You
NC-2	No PRDs exist for the Wedding and Feedback products	Their tracks (Sections 11-12) are skeletons until PRDs exist	Use Kit Stage 2 to create each PRD before building	You
NC-3	What your installed Codex plugins actually do and how they are invoked	Plugin names alone do not tell me	Prompt P00-A makes Codex list them	Codex, then you
NC-4	Who verifies the 40-destination data (costs, months, transit hubs)	PRD forbids fabricated or "verified" claims	Drafts from ChatGPT or Claude, you spot-check each row, `source_note` stays "Demo estimate; verification required"	You
NC-5	Current Gemini model id and free-tier quota	PRD §10 and §15 require verifying	Checked in Google AI Studio at step RS-14	You
NC-6	Is Playwright installed? Is Netlify CLI installed?	Tests and local function runs depend on them	P00-A checks. Missing ones need approved installs	Codex
NC-7	Source and licence of 40 destination images	PRD allows plain HTTPS URLs with fallback visuals	Image links tested individually; fallback shown when broken	You
NC-8	Product Space's full submission format	Only "live URL, demo video, PPT" is known	Section 18 checklist	You
NC-9	Whether each Codex thread runs locally or in a separate worktree by default in your app version	Decides whether parallel work is safe	Assume local and shared. Work sequentially	You (check in app)

2. My Tools, Environment and Constraints

Area	What you have	Notes for this playbook
Computer	Windows 11, HP Omen AI, Ryzen AI 7, RTX 5060, 24 GB RAM, 1 TB SSD	More than enough. The GPU is not needed. No local LLM is needed
Builder	OpenAI Codex Desktop (approval mode: Ask for approval)	Primary builder
Dev tools	Git, GitHub, Node.js, Python, Supabase, Netlify, Playwright "where available"	Verify before use (P00-A). Do not reinstall working tools
AI access	ChatGPT Plus with Codex, Gemini, Claude, Gemini API where free quota allows	Do not assume API credits come with any subscription
Budget	Rs 0 extra	Free tiers only. Check limits yourself: they change
Style	Prefers graphical interfaces, minimal copy-pasting, wants to understand each step	Netlify and Supabase through their web dashboards. Git through Codex's Git tools if present `[PUBLIC-DOCS]`

Where each tool is used

Tool	Use it for	Do not use it for
Claude (this chat)	Guiding you step by step, reviews, PRDs for Products 2 and 3, PPT and demo text, diagnosing errors	Writing the code files
ChatGPT or Gemini	Seed-data drafting, research, a second opinion	The same job as Claude in parallel (confusing)
Codex Desktop	All code, tests, Git, local runs	Deciding product scope
Supabase, Netlify, Google AI Studio dashboards	Database, hosting, API key	Anything that needs the key typed into a chat

Hard rules

- **Never paste an API key into Claude, ChatGPT, Codex or any file in the repo.** Keys go only in the Netlify dashboard (production) and a local `.env` file that is git-ignored.
- **Never claim a consumer subscription includes API quota.** Check in AI Studio.

3. Original Prompt Kit Audit and Mapping

3.1 Where Kit v1 and PRD v6.1 disagree (PRD v6.1 wins)

Topic	Kit v1 (Product 1)	PRD v6.1	Resolution
Gemini's job	Parse intent AND write "why this fits you" and a taste summary	Parse intent only. Rationales are deterministic templates; Gemini writes no blurbs	Remove kit prompts A2 and A3
Data tables	`user_intents`, `interactions`, `preference_weights`, `ai_cache`, `sessions`	One `destinations` table. Read-only to anonymous users. Feedback stays in browser memory	Remove all kit tables except `destinations`
Constraint handling	If too few results, relax the month and say so	Never relax budget or month silently. Show an empty state	Remove the relaxing rule
Tags	Fifteen-odd tags in the seed prompt	Exactly 14 canonical tags	Use the 14 in PRD §9
Scope	Travel with free-text plus chips, "learns" via stored weights	In-session weights only. Cross-visit memory is P1	Do not claim cross-visit learning
Catalog	30 India + 10 international	Up to 40 Indian destinations across six regions	Use PRD §12 plan
Schedule	1-1.5 days across four sprints	48-hour sprint, eight phases here	Replace with Section 9
Builder	Codex or Antigravity	Codex Desktop	Antigravity is a fallback only
GitHub	Optional at the start	Needed for Netlify deploys	Set up in Phase 1

3.2 Stage-by-stage audit (RoamSense)

Original prompt / stage	Decision	New location	Reason
Stage 1 Market scan	Keep (optional)	Phase 7, Claude or ChatGPT	Useful only for the PPT problem slide. Not on the build path
Stage 2 PRD builder	Remove for RoamSense; Keep for Products 2 and 3	Sections 11-12	PRD v6.1 is final. No new PRD cycle
Stage 3 PRD stress test	Revise	P00-B (spec-freeze check in Codex)	Becomes a consistency check, not a rewrite
Stage 4 Design pack	Revise	`docs/DESIGN.md`, step RS-05 and P02-D	PRD has states but no visual design. Lean design file needed
Stage 5 Seed data	Revise heavily	Track D, prompt P-SEED	New schema, 14 tags, six regions, verification step
Stage 6 Master build prompt	Replace	`AGENTS.md` (Section 8) plus thread prompts (Section 13)	Codex reads AGENTS.md. Short prompts per thread
Stage 7 Sprint follow-ups	Merge	Prompts P-FIX, P-HANDOFF, P-RESUME	Same purpose, now Codex-specific
Stage 8 AI prompts: A1 parse-intent	Revise	P01-C	Now uses PRD §10 want/avoid JSON and the 14 tags
Stage 8: A2 explain, A3 profile-summary	Remove	none	Conflict with PRD
Stage 9 Test and evaluator review	Revise and split	Section 14 (tests), Phase 7 (evaluator part)	v6.1-specific tests added
Stage 10 PPT, demo script, metrics	Keep	Phase 7, step RS-32	Still required deliverable
Appendix A troubleshooting	Merge	Section 17	Extended for Codex
Appendix B submission checklist	Merge	Section 18	Extended
"How to use" and 5-day schedule	Replace	Sections 1 and 9	Old schedule predates the 48-hour limit

3.3 Products 2 and 3

Original stage	Decision	Note
Stages 1-5, 9, 10	Keep	Reuse as written until each product's PRD exists
Stage 2 PRD builder	Keep, required	Products 2 and 3 have no PRD yet
Stage 6 Master build prompt	Replace later	Becomes AGENTS.md plus thread prompts once the PRD exists
Stage 8 AI prompts	Keep as drafts	Wedding B1-B3 and Feedback C1-C4 stay proposals until the PRDs confirm them

4. Installed Codex Skills and Plugin Strategy

4.1 What public documentation says

You asked me not to guess from package names. I looked up public pages for packages with these names. **I cannot see your installed copy.** Forks and versions differ, so P00-A makes Codex list exactly what is installed.

Package (your version)	What public docs describe [PUBLIC-DOCS]	Status
Ponytail (2.0.0)	A "lazy senior dev" ruleset that pushes the smallest working implementation and flags over-engineering. Skills named `ponytail`, `ponytail-review` (reviews a diff for over-engineering), plus audit, debt, gain and help in some listings. In Codex they are called with `@`	Unverified

4.2 Rules for using skills

1. **Never invoke all five on one task.** Each skill is used only where Section 4.3 says.
2. **Skills never override the PRD, AGENTS.md or the test rules.** A "minimal code" or "taste" instruction does not excuse skipping a required test or requirement.
3. **Design skills change styling only.** After any design pass, all tests must still pass and no behavior may change.
4. **Maximum two design passes** (P02-D and one polish before deployment). Do not use Impeccable and Taste in the same pass.
5. **Do not run installers or setup commands that edit** `AGENTS.md` (for example `graphify codex install`) unless you decide to.
6. **Invocation syntax differs between docs** (`@name`, `$name`, `/name`). Use the exact names P00-A reports.

4.3 Skill-to-Task Mapping Matrix

Task	Use this	When Codex should invoke	Do NOT invoke when	Fallback if missing
Product architecture	`spec-driven-dev`, `context-engineering`	Phase 0 freeze & Phase 4 integration	Mid-implementation	P00-B spells out checks
API contract	`api-and-interface-design`	Writing contract & `/api/discover`	Pure UI work	P00-D contract rules
Frontend dev	`frontend-ui-engineering`	Components, state, responsive	Backend files	P02-A requirements
Backend dev	`incremental-impl`, `TDD`	Ranking logic, intent parser, API	Styling	TDD steps in prompts
Database	none verified	n/a (SQL from PRD §12)	n/a	Section 15 Supabase
UI/UX refinement	Impeccable OR Taste	After P02-C passes tests	Before UI works / bugs	Skip. Core UI ok
Automated testing	`test-driven-development`	Every logic prompt (failing test first)	Trivial copy changes	Prompts require tests
Bug fixing	`test-driven-development`	Any logic bug (reproduce first)	Pure visual bug	P-FIX template
Security review	Check P00-A inventory	Phase 5	n/a	P05-B manual checklist
Deployment	none verified	n/a	n/a	Section 15 playbook
Documentation	none needed	n/a	n/a	P07-A
Over-engineering	`ponytail-review`	After Phases 2 & 3, on diff only	When it drops tests	Skip
Repo map	Graphify	Phase 4 if repo feels confusing	Default (repo is small)	Skip

5. Three-Product Sprint Overview

	Product 1: RoamSense	Product 2: Wedding Timeline	Product 3: Feedback Agent
Source of truth	**PRD v6.1** (final)	None yet [NC-2]	None yet [NC-2]
Assignment	Sprint I option 1	Sprint II	Sprint III option 3
Status	Ready to build	PRD needed first	PRD needed first
Data pattern	Read-only catalog, no writes	Writes (tasks, status). Needs auth model	CSV in browser. No DB needed
AI role	Parse free text into want/avoid tags	Explain risks & re-plan (kit proposal)	Cluster, root-cause, actions
Time budget	48 h max	Proposed 20-24 h	Proposed 20-24 h
Deliverables	Live URL, video, PPT	Live URL, video, PPT	Live URL, video, PPT

*The three products are **not** forced into one architecture. Reuse the process, not the schema.*

6. Product-Specific Source-of-Truth Documents

6.1 Priority order when documents disagree

> `PRD (per product)` > `docs/DECISIONS.md` > `AGENTS.md` > `Kit v1` > any prompt in this guide.

6.2 RoamSense repo files

File	Purpose	Owner
`docs/PRD_v6.1.md`	Authoritative requirements. Never edited by Codex	You
`docs/DESIGN.md`	Palette, type, spacing, card layout	You (Codex drafts)
`docs/DECISIONS.md`	Environment findings, spec observations, decisions	Thread 00
`docs/SPRINT_STATUS.md`	Where we are	Whoever finishes a task
`docs/TEST_RESULTS.md`	Real test output and dates	Thread 03 (others append)
`docs/DEPLOYMENT.md`	URLs, env var names (never values), smoke results	Thread 00
`AGENTS.md`	Standing rules for Codex	You, via thread 00

6.3 Spec observations on PRD v6.1 (non-blocking)

I read the PRD for contradictions. None blocks the build. These defaults get recorded in `docs/DECISIONS.md` at step RS-04. **None changes the PRD's intent.**

#	Observation	Default decision
SO-1	PRD §14 sends browser only to `/api/discover`. Never talks to Supabase	Netlify function uses Supabase **anon** key + RLS. No service-role key. Browser needs no Supabase keys
SO-2	`best_months` is a text array	Months are the exact strings `Jan` to `Dec`, validated in seed and request
SO-3	Orchha and Mandu are in MP but §12 lists them under North and West	PRD allows any one of the six values. Decide while curating. Only diversity tie-break uses region
SO-4	Debug footer (tag weights, unseen count) is visible in P0 per §6 and §13	Keep it, small and unobtrusive
SO-5	First-result target 2 s, Gemini timeout target 2,000 ms	PRD says targets, not guarantees. Make timeout configurable. Measure and record actual latency
SO-6	Avoid handling: "if `clean.length >= 3` return only clean"	Unit-test pool sizes 0, 1, 2, 3 and 4 so behavior is defined at edges
SO-7	Explanations must not claim live verification	Add a test: rationale text never contains "verified", "live", "guaranteed"
SO-8	Primary-tag weight applied to every destination with that tag in `vibe_tags`	Intended per PRD §9. Add a test that proves it
SO-9	"Up to 40" destinations, with fallback to smaller verified subset	Follow PRD §18: keep all six regions in any subset

7. Codex Desktop Setup and Conversation Architecture

7.1 Are five conversations the right structure?

Short answer: keep the five roles, run them as four threads, one at a time.

- Every new thread needs a handoff (copy-paste). You asked to minimize that.
- Deployment must happen early (Phase 1), so a separate "late" Deployment thread would sit idle for most of the sprint.
- So Deployment merges into the Lead thread, which already owns repo config and integration.
- If you prefer five, nothing breaks: split thread 00 into "Architecture" and "Deployment" and keep the same file ownership.

Thread name	Roles covered	Active in
RS-00 Lead	Architecture, integration, deployment, final consistency	Phases 0, 1, 4, 6, 7
RS-01 Backend	Supabase schema, seed validation, ranking logic, Gemini parser, `/api/discover`	Phase 2
RS-02 Frontend	UI, state, feedback logic, states, responsive	Phase 3
RS-03 QA	Browser tests, failure drills, security review, regression	Phase 5

7.2 Creating and naming threads

1. In Codex Desktop add your project folder `C:\Projects\roamsense` as a project. `[PUBLIC-DOCS]`
2. Start a new thread inside that project.
3. Rename it exactly as in the table above (for example `RS-01 Backend`). If you cannot find a rename option, put the name as the first line of your first message.
4. Menu labels may differ in your version. If an option is missing, tell me what you see and I will adapt the step.

7.3 How threads share work

- They share **files**, not memory. **Do not assume a thread can see another thread's chat.**
- Continuity comes from three files every thread reads first: `AGENTS.md`, `docs/SPRINT_STATUS.md`, `docs/DECISIONS.md`.
- Every thread ends its work by updating `docs/SPRINT_STATUS.md` and printing the handoff block (Section 16).

7.4 File ownership (prevents conflicting edits)

Path	Owner thread	Others may
`AGENTS.md`, `docs/**` (except tests), `package.json`, `vite.config.*`, `tsconfig*.json`, `netlify.toml`, `.gitignore`, `.env.example`, `shared/contract.ts`	RS-00 Lead	Read only. Ask Lead to change
`supabase/**`, `netlify/functions/**`, `shared/ranking.ts`, `shared/intent.ts`, `scripts/**`, `tests/unit/server/**`, `tests/api/**`	RS-01 Backend	Frontend may import `shared/*` only
`src/**`, `public/**`, `tests/unit/client/**`	RS-02 Frontend	Read only
`tests/e2e/**`, `docs/TEST_RESULTS.md`	RS-03 QA	Others append results

Rule: if a thread needs a change in a file it does not own, it stops and writes the request into `docs/SPRINT_STATUS.md` under "Requests". It never edits the file.

7.5 Sequential vs parallel

- **Default: sequential.** One thread works at a time. Finish, commit, then switch.
- **Parallel is safe only for work in different tools**, for example you curating seed data (Track D) while Codex scaffolds.
- **Parallel Codex threads need separate worktrees or branches.** [PUBLIC-DOCS] I do **not** assume your threads are isolated [NC-9]. Do not run two threads that edit files at once.
- **Advanced option (skip unless comfortable):** Frontend and Backend use different folders. If Codex lets you start Frontend in its own worktree, Lead merges afterwards. Otherwise stay sequential.

7.6 Approvals (Codex is set to "Ask for approval")

Routine. Approve after a quick glance: Creating or editing files in project folder; reading files; running `npm run test`, `npm run build`, `npm run dev`.

Pause and check each time:

Action	What to check before approving
Installing packages (`npm i`)	Package names are expected (react, vite, tailwindcss, vitest, playwright, @supabase/supabase-js). Beware of unfamiliar names
Applying migration	Matches PRD §12. Touches only `destinations` table. Run yourself in Supabase SQL editor
Changing RLS policies	Anonymous access is SELECT only. No INSERT, UPDATE or DELETE for anon
Editing environment variables	No real key appears in any file you could commit
Deleting files	Path is inside the project and you know why
Force-push or history rewrite	Say no. Not needed in this sprint
Publishing deployment	Tests passed first. You know which branch goes live
Cloud resource settings	Say no unless this guide asked for it

Never: approve a command you do not understand without asking me to explain it. Never ask Codex to bypass approvals or hide changes.

7.7 Starting every thread

Paste prompt **P-RESUME** (Section 13). It tells Codex to read `AGENTS.md`, `docs/SPRINT_STATUS.md` and `docs/DECISIONS.md` before doing anything.

8. AGENTS.md and Project Memory Strategy

8.1 Where it goes and how to confirm it works

- **Place:** `C:\Projects\roamsense\AGENTS.md` (repo root).
- **Why:** Codex uses `AGENTS.md` as persistent guidance [PUBLIC-DOCS]. It does not carry chat history between threads.
- **Create it:** with prompt P00-C (Section 13). Paste the template below.
- **Verify:** open a NEW thread and send: *"Without editing anything, list the section titles of the AGENTS.md instructions you loaded and the file path you read them from."* Expect the section titles below. If it cannot, check the file is at the repo root and the thread was opened in the project folder.
- **After editing AGENTS.md:** start a new thread so it loads the new version.

8.2 AGENTS.md template (RoamSense)

```
# AGENTS.md — RoamSense ## 1. Source of truth - Requirements: docs/PRD_v6.1.md. It is authoritative. Never edit it. - If this file, a prompt, or anything else conflicts with the PRD, stop and ask. Record the conflict in docs/DECISIONS.md. - Never change requirements, formulas or scope silently. ## 2. Scope control - Build P0 only. P1 and P2 items are forbidden unless the user explicitly approves. - Do not show UI controls for unimplemented features. - Do not add dependencies, features or files that the current task does not need. ## 3. Stack (do not change without approval) Vite, React, TypeScript, Tailwind CSS, Supabase PostgreSQL, Netlify Functions, Google Gemini API, Vitest (unit), Playwright (browser, only if installed). ## 4. Folders and ownership - src/**, public/** : frontend - netlify/functions/**, supabase/**, scripts/** : backend - shared/** : types and pure logic used by both - tests/unit/**, tests/api/**, tests/e2e/** : tests - docs/** : documentation Only edit paths owned by the current thread (stated in the prompt). If a change is needed elsewhere, write a request in docs/SPRINT_STATUS.md and stop. ## 5. Canonical contracts - Types, tag list, mood-to-tag map, budget tiers and months live in shared/contract.ts. Import them. Never redefine them. - The /api/discover request and response must match PRD sections 14 and 11 exactly. ## 6. Engineering rules - Write a failing test first, then the code, for every function with logic (use the test-driven-development skill if available). - Keep scoring and ranking as pure functions in shared/ so server and client use identical code. - Use other installed skills only where docs/DECISIONS.md allows. Design skills change styling only. - Prefer the simplest code that meets the PRD. Never remove a required test or behavior to make code smaller. ## 7. Testing and honesty - Run the relevant tests. Report the exact command and the actual output summary. - Clearly list: tests passed, tests failed, tests not run (and why). - Never say something works unless you ran it. Never fake or hide results. ## 8. Security - Secrets: GEMINI_API_KEY, GEMINI_MODEL, SUPABASE_URL, SUPABASE_ANON_KEY live only in environment variables. Never in code, logs, docs or commits. .env is git-ignored. .env.example has empty values. - The browser never receives secrets and never calls Supabase or Gemini directly. - Supabase: RLS on. Anonymous role has SELECT only on destinations. - Validate all request input on the server. Do not log raw free text. ## 9. Data integrity - Never invent destination data, costs, months, transit hubs, sources or "verified" claims. - Seed data comes only from the user's CSV. Rows keep source_note = 'Demo estimate; verification required' unless the user supplies a source. - UI text must not claim live prices, weather, safety or availability. ## 10. Approvals - Ask before installing packages, applying migrations, changing RLS, editing environment variables, deleting files, or deploying. - Never bypass approvals or hide significant changes. ## 11. Documentation and handoff - At the end of every task update docs/SPRINT_STATUS.md (current phase, completed, blocker, latest tests, next action, deployment status). - Record decisions in docs/DECISIONS.md and real test output in docs/TEST_RESULTS.md. - End every reply with the Standard Report: Done / Files changed / Tests run and results / Not done or blocked / Needs from user / Next step. - If blocked: stop, explain the blocker in plain English, and suggest one next action. Do not guess.
```

8.3 Project memory files

File	Updated when	Read when
`docs/SPRINT_STATUS.md`	End of every task	Start of every thread
`docs/DECISIONS.md`	Any decision or conflict	Start of every thread
`docs/TEST_RESULTS.md`	Every test run that matters	Before any gate check
`docs/DEPLOYMENT.md`	Every deployment	Phase 6 and when debugging live issues

Templates are in Section 16.

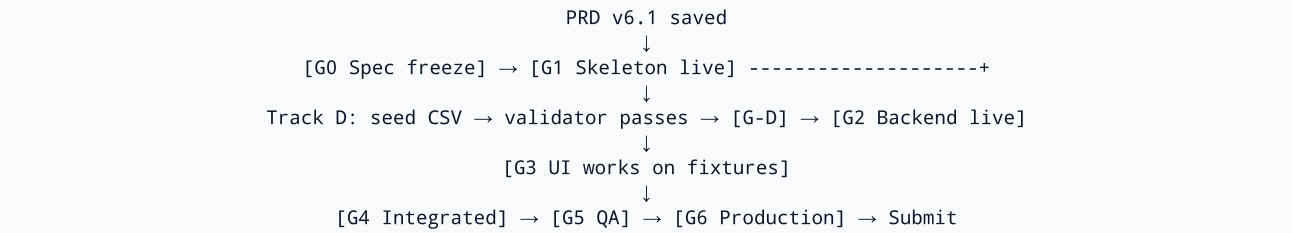
9. Master Sprint Roadmap and Dependency Map

9.1 RoamSense phases (H+ = hours after you start)

Phase	Hours	Effort	Owner	Depends on	Gate
0 Specification freeze	H+0 to 1.5	1.5 h	RS-00	PRD saved	**G0**
1 Foundation & skeleton	H+1.5 to 5	3.5 h	RS-00	G0	**G1**
Track D Seed data (parallel)	H+0 to 8	4-8 h	You	none	**G-D**
2 Data and backend	H+5 to 18	12 h	RS-01	G1, G-D (real import)	**G2**
3 Frontend	H+18 to 29	10 h	RS-02	G2 (contract fixed)	**G3**
4 Integration	H+29 to 33	4 h	RS-00	G3	**G4**
5 QA and hardening	H+33 to 40	7 h	RS-03	G4	**G5**
6 Deployment	H+40 to 44	4 h	RS-00	G5	**G6**
7 Submission	H+44 to 48	4 h	You, RS-00	G6	**Submitted**

These are planning targets, not guarantees. Your first run through each tool will be slower than my numbers.

9.2 Dependency map



Critical path: G1 → G2 → G3 → G4 → G5 → G6. **Biggest hidden risk:** Track D. If data is not ready by H+8, backend work continues on a fixture but you cannot pass G2.

9.3 P0 / P1 / P2 for RoamSense (from PRD §6)

- **P0 (must ship):** 40 curated destinations (or verified subset, six regions), four inputs plus optional free text, Gemini want/avoid parsing, hard budget and month gates, deterministic scoring with avoid handling, three cards plus a pool of up to 40, Like/Unlike/Dislike with replacement, exhaustion and empty states, debug footer, AI fallback, responsive UI, secrets protected, public URL, tests.
- **P1 (only after everything above is stable):** localStorage persistence, Saved drawer, Reset control.
- **P2 (not in this sprint):** accounts, live prices, itineraries.

9.4 De-scoping triggers

If this happens	Do this	Never do this
H+8 and < 25 verified destinations	Switch to smaller verified subset covering all six regions (PRD §18)	Invent rows to reach 40
H+18 and G2 not passed	Stop Frontend polish ideas. Finish `/api/discover` with Gemini fallback tested	Skip tests
H+29 and G3 not passed	Cut Debug footer polish & image niceties. Keep Like/Dislike logic	Remove Like/Dislike
H+40 and G5 not passed	Drop all P1 & optional design passes. Fix failing P0 tests only	Ship failing P0 tests
Gemini unavailable all day	Ship structured-only mode AND state plainly AI parsing was not demonstrated	Call it complete AI demo

Time-box rule: if one step takes 2x its estimate, stop and tell me. I will cut scope or change approach.
Minimum acceptable release: all PRD §19 items except any P1 item, with Gemini parsing working at least once on live URL.

9.5 Honest schedule check across all three products

- **Known:** the sprint was extended to 5 days (date unknown [NC-1]). RoamSense alone is planned at 48 hours.

10. RoamSense Step-by-Step Execution Track

How to read this section: Each row is one step. In guided mode I turn each row into the full format in Section 19. **Gate rule:** do not start next phase until gate passes. Commit and tag (e.g. `phase-2-pass`). **Rollback:** restore last tagged version.

Phase 0 — Specification Freeze (H+0 to 1.5) — RS-00 Lead

ID	Where	Do this	Done when
RS-01	Windows	Create `C:\Projects\roamsense` and `docs` inside. Paste PRD into Notepad. Save as `docs\PRD_v6.1.md` (All Files)	File opens and begins "RoamSense — Product Requirements Document (PRD) v6.1"
RS-02	Codex Desktop	Add folder as project. Create thread named `RS-00 Lead`	Thread shows project path
RS-03	RS-00	Send **P00-A** (read-only environment and skills check)	Reports Node, npm, Git versions, Playwright/Netlify CLI check, skill list. Nothing installed
RS-04	RS-00	Send **P00-B** (spec freeze)	`docs/DECISIONS.md` has SO-1 to SO-9 decisions & test table. PRD unchanged
RS-05	RS-00	Send **P00-C** (creates `AGENTS.md`, `docs/SPRINT_STATUS.md`, lean `docs/DESIGN.md`)	Three files exist. DESIGN.md has palette, font, spacing, card layout
RS-06	New thread	Open NEW thread. Send AGENTS.md verification question (§8.1)	Lists section titles. **Gate G0 passed**

Phase 1 — Foundation and Skeleton Deploy (H+1.5 to 5) — RS-00 Lead

ID	Where	Do this	Done when
RS-07	RS-00	Send **P00-D** (scaffold). Check package names before approving	`npm run dev` shows starter page. `npm run test` passes ≥1 test. `npm run build` succeeds
RS-08	GitHub & Codex	Create empty private GitHub repo. Ask Codex to connect & push	github.com shows files. No `.env` in repo
RS-09	Netlify web	**P00-E** gives settings. Build `npm run build`, pub `dist`, func `netlify/functions`	Public URL shows starter. `/api/health` returns ok JSON. **Gate G1 passed.** Tag `phase-1-pass`

Track D — Seed data (you, parallel, H+0 to 8)

ID	Where	Do this	Done when
D-1	ChatGPT/Claude	Send **P-SEED**. Drafts CSV from PRD §12 list	CSV with columns exactly as in PRD §12
D-2	Browser	Spot-check cost range, best months, nearest transit hub. Fix errors. Keep `source_note` unless real source added	Every row is checked or left with default note
D-3	Browser	Find HTTPS image URL per destination. Test each link	Each link loads. Broken ones replaced or left for fallback
D-4	Windows	Save as `supabase\seed\destinations.csv` in project	File in place
D-5	After RS-10	Codex validator passes (RS-10)	**Gate G-D passed:** 40 rows pass, or verified subset across six regions

Phase 2 — Data and Backend (H+5 to 18) — RS-01 Backend

ID	Where	Do this	Done when
RS-10	Thread `RS-01 Backend`	Send **P-RESUME** then **P01-A** (SQL migration, validator, fixture)	Validator tests pass. Migration file matches PRD §12 exactly
RS-11	Supabase website	Create project. Run SQL migration. Import CSV in Table Editor. Confirm RLS on	Table shows rows. `scripts/check-rls` proves anon SELECT works and INSERT denied
RS-12	RS-01	Send **P01-B** (ranking logic, TDD)	All ranking unit tests pass (pool sizes 0-4, tie-breaks)
RS-13	RS-01	Send **P01-C** (Gemini intent parser with mocks)	Parser tests pass for valid, malformed, wrong-type, out-of-voc, timeout
RS-14	Google AI Studio	[YOUR ACTION] Create key. Check model names & quota. Put key in local `.env`	Model confirmed. `.env` is git-ignored
RS-15	RS-01	Send **P01-D** (`/api/discover`)	API tests pass. Local request returns contract JSON. Fallback works
RS-16	RS-00/01 & Netlify	Push. In Netlify add env vars. Redeploy	Live `/api/discover` returns rows. **Gate G2 passed.** Tag `phase-2-pass`

Phase 3 — Frontend (H+18 to 29) — RS-02 Frontend

ID	Where	Do this	Done when
RS-17	Thread `RS-02 Frontend`	Send **P-RESUME** then **P02-A** (form, validation, skeleton)	Form shows PRD error text on missing input. Duplicate submit blocked
RS-18	RS-02	Send **P02-B** (cards, Like/Unlike/Dislike, replacement)	Feedback unit tests pass. Like, Unlike, Dislike, exhaustion work on fixture
RS-19	RS-02	Send **P02-C** (empty, error, AI notice, avoid notice, fallback, mobile)	All states visible. Page works at 375 px wide and desktop
RS-20	RS-02	**Optional.** Send **P02-D** (one design pass) only if RS-19 passes	Tests still pass. No behavior changed. **Gate G3 passed.** Tag `phase-3-pass`

Phase 4 — Integration (H+29 to 33) — RS-00 Lead

ID	Where	Do this	Done when
RS-21	RS-00	Send **P04-A** (connect real API, contract check)	App shows real destinations. Contract test passes
RS-22	Browser	Walk through 5 scenarios (§14). Record in `docs/TEST_RESULTS.md`	Results recorded honestly. **Gate G4 passed.** Tag `phase-4-pass`

Phase 5 — QA and Hardening (H+33 to 40) — RS-03 QA

ID	Where	Do this	Done when
RS-23	Thread `RS-03 QA`	Send **P-RESUME** then **P05-A** (browser journeys)	Journeys run. Failures listed with reproduction
RS-24	RS-03	Send **P05-B** (security review)	No secrets in repo/dist. RLS confirmed. Server validation ok
RS-25	RS-03	Send **P05-C** (failure drills: timeout, bad key, bad model, DB error)	Each drill matches PRD expected behavior
RS-26	Owner threads	Fix failures in owning thread using **P-FIX**	Re-run shows fixed. **Gate G5 passed.** Tag `phase-5-pass`

Phase 6 — Deployment (H+40 to 44) — RS-00 Lead

ID	Where	Do this	Done when
RS-27	RS-00 & Netlify	Send **P06-A** (final deploy checklist & smoke script)	Live smoke tests pass. Env var names match `docs/DEPLOYMENT.md`
RS-28	Your phone	[YOUR ACTION] Open public URL on real phone. Run full journey	Works without critical errors. Screenshot taken. **Gate G6 passed**

Phase 7 — Submission (H+44 to 48)

ID	Where	Do this	Done when
RS-29	RS-00	Send **P07-A** (README and evidence list)	README exists. No claim beyond test proofs
RS-30	Real people	[YOUR ACTION] Run sessions with up to 5 testers (PRD §16)	Numbers from real sessions. State tester count honestly
RS-31	Claude	Run Kit Stage 10 (PPT outline, 2-min demo script, metrics note)	Outline & script ready. Unknowns: [ADD REAL NUMBER]
RS-32	Windows	Screen-record live app while reading script	Video file exists
RS-33	Section 18	Tick the final checklist	All items ticked or honestly marked

11. Product 2 (Wedding Timeline Orchestrator) — Execution Track

Status: skeleton. No PRD exists [NC-2]. Nothing below is a requirement until your PRD confirms it.

11.1 Gate 0 — Create the PRD (before any code)

ID	Where	Do this	Done when
W-01	Claude (new chat)	Run Kit **Stage 2** (Wedding version)	Complete PRD with numbered requirements
W-02	New chat	Run Kit **Stage 3** review. Accept cuts	PRD updated
W-03	Windows	Save as `docs\PRD_v1.md` in `C:\Projects\wedding`	File saved
W-04	Codex	Create `AGENTS.md` adapting §8.2 (tables, API, write auth)	Verified in a new thread

11.2 Decisions the PRD must settle (they change the architecture)

- **How writes are secured:** Saves tasks and status. Anonymous read-only will not work. Options: writes through Netlify Function using server secret, or RLS tied to share token. Pick one in PRD.
- **What AI may do:** Explain next steps & risks; propose re-plans for approval. Never change dates/statuses on its own.
- **Deterministic engine:** Due dates, dependency blocking, date shifting, crunch flags computed in pure code & tested.

11.3 Phases (lean)

Phase	Target	Thread
0-1 Freeze, skeleton deploy	2 h	W-00 Lead
2 Data and engine (templates, dependency logic, date-shift tests)	7 h	W-01 Backend
3 Frontend (create wedding, timeline, task detail, this week, share view)	6 h	W-02 Frontend
4-5 Integration and QA	4 h	W-00, W-03
6-7 Deploy and submit	3 h	W-00, you

Test checklist & deliverables (§11.4-11.5): 4-month offset test; late prerequisite shifts dependents; circular dependency rejected; crunch flag triggers; share link is strictly read-only; re-plan saves nothing before approval; AI failure leaves engine working. Deliverables: live URL, video, PPT, README.

12. Product 3 (Product Feedback to Insight Agent) — Execution Track

Status: skeleton. No PRD exists [NC-2]. Nothing below is a requirement until your PRD confirms it.

12.1 Gate 0 — Create the PRD (before any code)

ID	Where	Do this	Done when
F-01	Claude (new chat)	Run Kit **Stage 2** (Feedback agent version)	Complete PRD
F-02	New chat	Run Kit **Stage 3** review	PRD updated
F-03	Windows	Save as `docs\PRD_v1.md` in `C:\Projects\feedback-agent`	File saved
F-04	Codex	Create `AGENTS.md` from §8.2, adapted	Verified in a new thread

12.2 Decisions the PRD must settle

- **Do you need a database at all?** CSV can be parsed in browser and analyzed per session. Skipping DB saves hours and avoids storing customer text. If storing data, define privacy and RLS.
- **Who computes numbers:** Code counts items, percentages, priority scores. AI only groups items & drafts hypotheses/actions. Every theme links to evidence rows.
- **Sample size & confidence:** Define rules (kit proposal: low confidence under 30 rows or <5 items/theme).
- **Function timeouts:** Plan batching in PRD. **Privacy:** Synthetic/anonymized data only.

12.3 Phases (lean)

Phase	Target	Thread
0-1 Freeze, skeleton deploy	2 h	F-00 Lead
2 Logic (CSV validation, scoring, caveats, evidence linking) & AI functions	8 h	F-01 Backend
3 Frontend (upload, data check, ranked themes, theme detail with evidence)	5 h	F-02 Frontend
4-5 Integration & QA using answer-key CSV	4 h	F-00, F-03
6-7 Deploy and submit	3 h	F-00, you

Test checklist & deliverables (§12.4-12.5): Demo CSV themes match answer key; loud-but-small theme ranks below quiet-widespread theme; 20-row file shows low-confidence warnings; broken CSV shows friendly error; evidence rows text matches; no number on screen comes from AI. Deliverables: live URL, video, PPT, README.

13. Optimized Codex Prompt Library (RoamSense)

How these prompts work: Each prompt covers Objective, Context, Scope, Non-goals, Skills, File ownership, Implementation, Tests, Acceptance, Expected output. Paste **[FOOTER]** at the bottom of every prompt:

```
[FOOTER] If blocked: stop, explain the blocker in plain English, suggest ONE next action, and do not guess or work around the PRD. When finished: update docs/SPRINT_STATUS.md and end with the Standard Report: Done / Files changed / Tests run and results (passed, failed, NOT run and why) / Not done or blocked / Needs from user / Next step. Then print the handoff block from AGENTS.md section 11.
```

P-RESUME — Start of every new thread

```
Resume work on RoamSense. Before doing anything: 1. Read AGENTS.md, docs/SPRINT_STATUS.md and docs/DECISIONS.md. 2. Do not edit any file yet. 3. Reply with: current phase, what is complete, the current blocker (if any), the files this thread owns, and the next action. Then wait for my instruction.
```

P00-A — Environment and skills check (RS-03, thread RS-00)

```
OBJECTIVE: Verify my development environment and installed Codex skills. Read-only except for one documentation write. CONTEXT: Windows 11 laptop. I must not reinstall working tools. I need exact facts, not assumptions. SCOPE: 1. Report versions of Node, npm and Git. Use only commands that cannot download or install anything. 2. Report whether Playwright and Netlify CLI are already installed. Do NOT run npx for anything that might be missing, because npx can download it. Use "where" and "npm ls -g --depth=0" instead. 3. List every plugin and skill you can see. For each: exact name, exact way I should invoke it in this app, a one-line description taken from its own documentation, and whether it can write files at the repo root or edit AGENTS.md. Do not run any of them. 4. State the approval mode and sandbox mode if visible to you. 5. Create docs/DECISIONS.md with an "Environment" section containing these findings. Mark anything you could not verify as UNVERIFIED. NON-GOALS: No installs. No project files. No code. SKILLS: none. FILE OWNERSHIP: docs/DECISIONS.md only. TESTS: none.
```

P00-B — Specification freeze (RS-04, thread RS-00)

```
OBJECTIVE: Freeze the specification. Find contradictions or gaps in docs/PRD_v6.1.md and turn the PRD into a test-case table. Do not change the PRD and do not write code. CONTEXT: PRD v6.1 is authoritative. Sections 8 to 14 and 19 matter most. SCOPE: 1. Read the whole PRD. List contradictions or ambiguities. For each give the PRD section, the issue, and your proposed default. Do not resolve any by editing the PRD. 2. Record these decisions in docs/DECISIONS.md exactly as stated: SO-1 The Netlify function uses the Supabase anon key plus RLS. No service-role key anywhere. The browser needs no Supabase keys. SO-2 Months are the exact strings Jan to Dec. SO-3 Region for borderline places is chosen during curation. Only the diversity tie-break uses region. SO-4 The debug footer is shown in P0 as the PRD says. SO-5 The Gemini timeout is configurable through GEMINI_TIMEOUT_MS. Latency is measured, not promised. SO-6 Avoid handling is unit-tested for pool sizes 0, 1, 2, 3 and 4. SO-7 Rationale text must never contain the words verified, live or guaranteed. SO-8 The primary-tag weight applies to every destination with that tag in vibe_tags; test it. SO-9 If fewer than 40 verified destinations exist, use a smaller verified subset covering all six regions (PRD section 18). SO-10 matched_tags = vibe_tags intersected with the union of mood tags and want tags, in canonical tag order. If empty, show General Match. SO-11 companion_fit values are Solo, Partner and Friends, the same as the form values. 3. Build a table that maps every requirement in FR-1 to FR-5 and PRD sections 9 to 11 to test IDs and test level (unit, api, e2e, manual). NON-GOALS: Do not edit docs/PRD_v6.1.md. Do not add features. Do not write code. SKILLS: spec-driven-development (if available). FILE OWNERSHIP: docs/DECISIONS.md only. TESTS: none yet. ACCEPTANCE: DECISIONS.md lists decisions SO-1 to SO-11, open questions, and the test-case table. "git diff docs/PRD_v6.1.md" shows no changes. EXPECTED OUTPUT: Summary of contradictions found (or "none") and the number of test cases. [FOOTER]
```

P00-C — AGENTS.md, status file and lean design file (RS-05, thread RS-00)

```
OBJECTIVE: Create the project instruction and tracking files. CONTEXT: AGENTS.md must be exactly the text I paste below. docs/DESIGN.md is a lean visual guide for a mobile-first travel discovery app. SCOPE: 1. Create AGENTS.md at the repo root with exactly this content: [PASTE THE AGENTS.md TEMPLATE FROM SECTION 8.2] 2. Create docs/SPRINT_STATUS.md with sections: Current product, Current phase, Completed, Current blocker, Latest tests, Next action, Deployment status, Remaining critical work, Requests. Fill: product RoamSense, phase 0. 3. Create docs/DESIGN.md (under 60 lines): a calm palette with hex codes (primary, secondary, background, text, success, warning amber), one readable web-safe or system font stack, spacing scale, card layout (hero image, name and state, cost line with disclosure, tags, rationale, Like and Dislike), button style, loading skeleton, empty and error state tone. Friendly, non-corporate tone. No heavy animation. NON-GOALS: No code. Do not modify the PRD. SKILLS: none required. FILE OWNERSHIP: AGENTS.md, docs/SPRINT_STATUS.md, docs/DESIGN.md. TESTS: none. ACCEPTANCE: Three files exist. AGENTS.md matches the pasted text. EXPECTED OUTPUT: Confirmation plus the first 10 lines of each file. [FOOTER]
```

P00-D — Scaffold project and contract (RS-07, thread RS-00)

```
OBJECTIVE: Create the project skeleton and the shared contract. No product features yet. CONTEXT: PRD sections 9, 12 and 14. Stack in AGENTS.md section 3. Ask me before every dependency install. SCOPE: 1. Scaffold Vite + React + TypeScript + Tailwind CSS in this folder without overwriting docs/ or AGENTS.md. 2. Add Vitest and the scripts: dev, build, test. 3. Create folders: src/, shared/, netlify/functions/, supabase/migrations/, supabase/seed/, scripts/, tests/unit/server/, tests/unit/client/, tests/api/, tests/e2e/. 4. Create shared/contract.ts as the single source for: the 14 canonical tags, the four moods and their tag lists (PRD section 9), companions (Solo, Partner, Friends), budget tiers and the cascade rule, months (Jan to Dec), regions (six), and TypeScript types for Destination, Candidate, DiscoverRequest and DiscoverResponse exactly as in PRD section 14. Types and constants only. No logic. 5. Create netlify/functions/health.ts returning {"ok": true}. Create netlify.toml: build command "npm run build", publish "dist", functions "netlify/functions", and a redirect from /api/* to /.netlify/functions/:splat. 6. Create .env.example with empty values for SUPABASE_URL, SUPABASE_ANON_KEY, GEMINI_API_KEY, GEMINI_MODEL, GEMINI_TIMEOUT_MS. 7. Create .gitignore covering node_modules, dist, .env, .env.* except .env.example, .netlify, test-results, playwright-report, .impeccable, graphify-out. 8. Add a first test (write it first): the tag list has 14 unique entries and every mood tag is in the tag list. NON-GOALS: No UI beyond the default starter page. No Supabase or Gemini code. No Playwright install yet. SKILLS: api-and-interface-design for the contract. test-driven-development for the first test. FILE OWNERSHIP: all root config files, shared/contract.ts, netlify/functions/health.ts. TESTS: npm run test, npm run build. ACCEPTANCE: npm run dev shows a page. npm run test passes. npm run build passes. No secrets in any file. EXPECTED OUTPUT: The three command results and the file list. [FOOTER]
```

P00-E — Deployment notes (RS-09, thread RS-00)

```
OBJECTIVE: Write docs/DEPLOYMENT.md with click-by-click Netlify setup for this repo, then wait while I do it. CONTEXT: Repo already has netlify.toml. I use the Netlify website, not the command line. I am a beginner. SCOPE: 1. List the exact Netlify settings: import from GitHub, build command, publish directory, functions directory. 2. List environment variable NAMES only (never values): SUPABASE_URL, SUPABASE_ANON_KEY, GEMINI_API_KEY, GEMINI_MODEL, GEMINI_TIMEOUT_MS. Say that functions need them available at runtime and that changing them requires a redeploy. 3. Add a smoke-test list: home page loads, /api/health returns ok. 4. Add a table for me to fill in: live URL, last deploy date, last smoke result. NON-GOALS: Do not deploy. Do not touch any account. FILE OWNERSHIP: docs/DEPLOYMENT.md. SKILLS: none. TESTS: none. ACCEPTANCE: Steps are specific enough for a first-time Netlify user. EXPECTED OUTPUT: Confirmation. [FOOTER]
```

P-SEED — Seed data draft (Track D, run in ChatGPT or Claude, not Codex)

```
Create demo seed data for RoamSense, a curated India-wide travel discovery app. Output a CSV with these exact columns: name, state, region, budget_tier, best_months, vibe_tags, primary_vibe_tag, companion_fit, estimated_cost_lower_inr, estimated_cost_upper_inr, indicative_cost_range, typical_duration, nearest_transit_hub, description, image_url, source_note, csv_index Destinations (40): North: Landour, Bir Billing, Tirthan Valley, Spiti, Leh, Rishikesh, Gulmarg, Orchha. South: Gokarna, Coonoor, Vagamon, Varkala, Hampi, Chikmagalur, Kodaikanal, Wayanad, Dandeli. West: Udaipur, Jaisalmer, Mandu, Rann of Kutch, Alibaug, Jawai. East: Darjeeling, Puri, Kalimpong, Digha, Shantiniketan. Northeast: Ziro, Mawlynnong, Cherrapunji, Tawang, Majuli, Dzukou Valley. Central: Pachmarhi, Bandhavgarh, Kanha, Khajuraho, Bhedaghat, Pench. Rules: - region is exactly one of North, South, East, West, Central, Northeast. - vibe_tags and primary_vibe_tag use ONLY these 14 tags: misty, mountains, beach, heritage, quiet, cafes, nightlife, adventure, trekking, spiritual, wildlife, nature, scenic, culture. primary_vibe_tag must be one of that row's vibe_tags. Use pipe | between multiple values. - best_months uses Jan to Dec, pipe-separated. - companion_fit uses Solo, Partner, Friends, pipe-separated. - Cost basis: estimated INR per person for three nights of private-room accommodation and food, excluding transport to and from the destination. budget_tier is decided by the UPPER bound: up to 10000 = budget; 10001 to 25000 = mid; above 25000 = premium. These are illustrative estimates. - source_note for every row: Demo estimate; verification required - csv_index runs 1 to 40. - image_url: leave blank. I will find and test links myself. - Do NOT invent sources, ratings or claims. Where you are unsure of a value, give your best estimate and add [CHECK] at the end of that row's description so I can find it. - description: one factual sentence under 25 words, no superlatives. Afterwards list the 10 rows you are least sure about.
```

P01-A — Schema, seed validator, fixture (RS-10, thread RS-01)

```
OBJECTIVE: Create the database migration, a seed validator, an RLS check, and a small fixture. CONTEXT: PRD section 12 (schema and rules), section 15 (security). Seed CSV is supplied by me at supabase/seed/destinations.csv. SCOPE: 1. Create supabase/migrations/001_destinations.sql using the exact table, constraints and RLS policy in PRD section 12. Do not change column names or constraints. 2. Write the failing tests first, then scripts/validate-seed.ts, which fails with readable messages when: row count is over 40, a region is not one of six, a tag is outside the 14, primary_vibe_tag is not in vibe_tags, a month is not Jan to Dec, budget_tier does not match the upper cost bound (boundaries 10000, 10001, 25000, 25001), csv_index is duplicated, image_url is not https (blank is allowed and reported), lower cost exceeds upper, or any of the six regions has no rows. Warn if a description contains [CHECK]. 3. Create tests/fixtures/discover.sample.json: 8 clearly fake demo candidates (names like "Demo Hill A"), matching DiscoverResponse in PRD section 14, marked demo. Used by the frontend until integration. 4. Create scripts/check-rls.ts that, using SUPABASE_URL and SUPABASE_ANON_KEY from .env without printing them, proves anonymous SELECT works and anonymous INSERT, UPDATE and DELETE are denied. Do not run it until I have run the migration. NON-GOALS: Do not invent or edit seed rows. Do not run the migration yourself. No API code. SKILLS: test-driven-development. FILE OWNERSHIP: supabase/**, scripts/**, tests/unit/server/**, tests/fixtures/**. TESTS: write validator tests with small bad-row examples for each rule, run them. ACCEPTANCE: Validator tests pass. If my CSV exists, report the validator result on it honestly, including failures. EXPECTED OUTPUT: Test results, validator result on the real CSV, and the exact steps for me to run the migration in the Supabase website. [FOOTER]
```

P01-B — Ranking logic (RS-12, thread RS-01)

```
OBJECTIVE: Implement the deterministic eligibility, scoring, avoid handling, ordering, and rationale logic as pure functions. CONTEXT: PRD sections 2 (FR-2, FR-3), 9, 11. DECISIONS.md SO-6 to SO-11. This code is shared by server and client. SCOPE: Create shared/ranking.ts exporting pure functions for: 1. eligibility: budget cascade (budget sees budget; mid sees budget and mid; premium sees all) and month must be in best_months. Never relax either gate. 2. base_score per PRD section 11 step 2. 3. avoid handling per PRD section 11 step 3, including avoid_notice. 4. ordering per step 4: score descending; on ties prefer a region not already represented among currently selected visible cards; then csv_index ascending. Diversity never outranks a higher score. Export one comparator that both server and client will use. 5. matched_tags (SO-10) and the rationale template from PRD FR-3, which must never contain the words verified, live or guaranteed. 6. current_score = base_score + the sum of tag weights across the destination's vibe_tags (PRD section 9). NON-GOALS: No network calls. No UI. No changes to PRD formulas. SKILLS: test-driven-development (failing test first for every rule), incremental-implementation. After finishing, run ponytail-review on the diff only if available, and do not accept any suggestion that removes a test or a PRD behavior. FILE OWNERSHIP: shared/ranking.ts, tests/unit/server/ranking.test.ts. TESTS: Hand-calculate at least 6 expected scores. Cover budget boundaries and cascade, month eligibility, pool sizes 0 to 4 with avoid tags, avoid penalty of -4 per match, a tag that appears in both mood tags and avoid, tie-break by region then csv_index, diversity not beating a higher score, rationale wording, matched_tags order, and General Match when empty. ACCEPTANCE: All tests pass. No function reads the network, clock or randomness. EXPECTED OUTPUT: Test summary and the list of exported functions. [FOOTER]
```

P01-C — Gemini intent parser (RS-13, thread RS-01)

```
OBJECTIVE: Implement the server-side Gemini intent parser with strict validation and fallback. CONTEXT: PRD sections 5 (FR-5), 10 and 15. The model name comes from GEMINI_MODEL. The key comes from GEMINI_API_KEY. Never log free text or keys. SCOPE: Create shared/intent.ts (pure validation and normalization) and netlify/functions/lib/gemini.ts (the call). 1. Use the system prompt in PRD section 10 exactly. 2. Skip the call entirely when free text is empty or only whitespace. 3. Timeout from GEMINI_TIMEOUT_MS (default 2000). 4. Validation: require an object with two string arrays "want" and "avoid". Reject malformed JSON or wrong types. Lowercase, de-duplicate, remove tags outside the 14. If a tag appears in both lists, remove it from want (avoid wins). 5. Return free_text_applied=true only after a valid parse. On timeout, invalid output, missing key or missing model, return empty want and avoid with free_text_applied=false and a reason code. Never throw to the caller. 6. Do not hard-code a model name. NON-GOALS: No retries. No caching. Gemini writes no blurbs or facts. SKILLS: api-and-interface-design, test-driven-development. FILE OWNERSHIP: shared/intent.ts, netlify/functions/lib/gemini.ts, tests/unit/server/intent.test.ts. TESTS (mock fetch, never call the real API in tests): valid response; markdown-fenced JSON; malformed JSON; wrong field types; out-of-vocabulary tags; duplicates and mixed case; tag in both lists; timeout; HTTP error; missing key; missing model; empty and whitespace text skip the call. ACCEPTANCE: All tests pass. A grep of the code finds no hard-coded key or model name. EXPECTED OUTPUT: Test summary. If I have put a real key in .env, tell me how to run a manual one-time smoke call and wait for my approval first. [FOOTER]
```

P01-D — /api/discover (RS-15, thread RS-01)

```
OBJECTIVE: Implement POST /api/discover per PRD sections 8, 11 and 14. CONTEXT: Uses shared/ranking.ts and shared/intent.ts. Supabase accessed with SUPABASE_URL and SUPABASE_ANON_KEY on the server only (SO-1). SCOPE: 1. Validate the request: companions, mood, budget and month must be allowed values from shared/contract.ts. free_text is optional, trimmed, maximum 500 characters. Return a clear 400 error for invalid input. 2. Run the Supabase query (budget cascade and month) and the Gemini parse concurrently using Promise.allSettled. A Gemini failure must not fail the request. A database failure returns a clear error response that the UI can distinguish from an AI failure. 3. Apply avoid handling, scoring and ordering from shared/ranking.ts. Return up to 40 candidates with every field the client needs (matched_tags, rationale, base_score, csv_index). 4. Response matches PRD section 14: success, free_text_applied, user_tags (mood_tags, want, avoid), avoid_notice, candidates. 5. Never log raw free text, keys or full request bodies. 6. Return an empty candidates list (not an error) when nothing matches. NON-GOALS: No writes to the database. No caching. No accounts. SKILLS: api-and-interface-design, test-driven-development. FILE OWNERSHIP: netlify/functions/discover.ts, tests/api/**. TESTS (mock Supabase and Gemini): valid request; each invalid field; free text over 500; empty free text means no Gemini call; Gemini failure still returns structured results with free_text_applied=false; database failure returns the distinct error; every returned candidate satisfies budget and month; zero matches returns an empty list; candidates never exceed 40; the response validates against the contract types. ACCEPTANCE: All tests pass. A local sample request returns JSON matching the contract (explain how to run it locally; ask before installing Netlify CLI if it is missing). EXPECTED OUTPUT: Test summary and a sample response with fake or real rows clearly labeled. [FOOTER]
```

P02-A — Form, validation and loading (RS-17, thread RS-02)

```
OBJECTIVE: Build the input experience against tests/fixtures/discover.sample.json. No real API yet. CONTEXT: PRD sections 7, 8 (FR-1), 13, 15. Design guide: docs/DESIGN.md. Import all types and constants from shared/contract.ts. SCOPE: 1. Header "RoamSense / Explore Curated Indian Getaways" and the disclosure "Demo Dataset: Up to 40 Curated Indian Destinations." 2. Exactly-one-choice selectors for companions, mood, budget tier and month. Optional free-text box limited to 500 characters with a short note that the text is sent to an external AI service for parsing (PRD section 15). 3. Discover button. If any selection is missing show: "Please select companions, mood, budget tier, and travel month." 4. Disable duplicate submissions while a request is pending. Show three skeleton cards while loading. 5. A data-access layer in src/lib/api.ts with one function that returns the fixture for now, shaped like the real response, so integration only swaps this function. 6. Mobile-first layout. NON-GOALS: No Saved drawer, no Reset button, no P1 controls. No calls to Supabase or Gemini. SKILLS: frontend-ui-engineering. FILE OWNERSHIP: src/**, public/**, tests/unit/client/**. TESTS: component or unit tests for validation message, disabled-while-pending, and the 500-character limit. Run them. ACCEPTANCE: Tests pass. npm run dev shows the form working at 375 px width. EXPECTED OUTPUT: Test summary and a short description of what I should click to check. [FOOTER]
```

P02-B — Cards and feedback (RS-18, thread RS-02)

```
OBJECTIVE: Build the result cards and the Like, Unlike and Dislike behavior. CONTEXT: PRD sections 7, 8 (FR-3, FR-4), 9, 11. Use the comparator and score functions from shared/ranking.ts. Do not re-implement scoring. SCOPE: 1. Show up to three cards with: name, state, hero image or fallback, estimated cost range with this exact kind of disclosure (estimated INR per person for three nights of private-room stay and food, excluding transport, illustrative not live), typical duration, nearest transit hub, matched-tag badges or General Match, the rationale, Like and Dislike controls. 2. Pure state logic in src/lib/feedback.ts, derived from liked IDs and dismissed IDs so toggling is idempotent: Like adds +2 to the card's primary tag weight (filled heart); Unlike removes exactly that contribution; each liked card counts independently even when tags overlap; Dislike dismisses the card for this search, adds -2 to its primary tag, and fills that slot with the highest-ranked eligible unseen candidate. Other visible cards stay in their slots. Dismissed cards never return. No network request on feedback. 3. When no unseen candidate remains show: "No more matching destinations in this criteria pool". 4. A new search clears likes, dislikes, weights and dismissed IDs. 5. Debug footer showing parser status, active tag weights and the unseen count. NON-GOALS: No persistence across visits. No Saved drawer. No Reset button. SKILLS: frontend-ui-engineering, test-driven-development for src/lib/feedback.ts. FILE OWNERSHIP: src/**, tests/unit/client/**. TESTS: Like then Unlike returns weights to zero; two likes on cards sharing a tag; Dislike weight -2 and replacement choice; replacement tie-break; other cards keep their slots; exhaustion message; new-search reset; weights recomputed from state give the same result as incremental updates. ACCEPTANCE: Tests pass. On the fixture every behavior works by clicking. EXPECTED OUTPUT: Test summary and the click path to verify each behavior. [FOOTER]
```

P02-C — States and responsive polish (RS-19, thread RS-02)

```
OBJECTIVE: Finish all UI states and responsiveness. CONTEXT: PRD sections 5 (FR-5), 8, 13. Design guide: docs/DESIGN.md. SCOPE: 1. Parsed-intent banner when free_text_applied is true. 2. Amber dismissible notice when free text was provided but not applied. 3. The avoid-warning text from PRD section 11 when avoid_notice is true. 4. Empty state with an actionable message when no candidates match (never silently relax budget or month). 5. Separate error states for database or network failure. 6. Image fallback visual when an image fails to load. 7. Accessibility basics: labels on inputs, keyboard focus visible, sufficient contrast, buttons have text or aria-labels. 8. Check layouts at 375 px, 768 px and desktop widths. NON-GOALS: No new features. No P1 controls. SKILLS: frontend-ui-engineering. FILE OWNERSHIP: src/**, tests/unit/client/**. TESTS: component tests for each state. Run them. ACCEPTANCE: Every state can be triggered from the fixture or a test switch. No horizontal scrolling at 375 px. EXPECTED OUTPUT: Test summary and a list of how to trigger each state. [FOOTER]
```

P02-D — One design pass (RS-20, optional, thread RS-02)

```
OBJECTIVE: One visual refinement pass. Styling only. CONTEXT: All tests currently pass. docs/DESIGN.md is the direction. SCOPE: Use ONE installed design skill: Impeccable (audit, then polish) OR Taste, not both. Do not run any setup or "teach" command that writes PRODUCT.md or DESIGN.md at the repo root. If the skill creates working folders, confirm they are git-ignored. NON-GOALS: No behavior, logic, copy meaning, or dependency changes. No heavy animation or new libraries. SKILLS: the one chosen design skill only. FILE OWNERSHIP: src/** styling files only. TESTS: run the full test suite before and after. ACCEPTANCE: Same test results as before. Layout still works at 375 px. List every changed file. EXPECTED OUTPUT: Before and after test results and a summary of visual changes. [FOOTER]
```

P04-A — Integration (RS-21, thread RS-00)

```
OBJECTIVE: Connect the frontend to the real /api/discover and prove front and back agree. CONTEXT: PRD section 14. The only change to frontend code is the body of the function in src/lib/api.ts. SCOPE: 1. Replace the fixture call with a real POST to /api/discover. 2. Add a contract test that validates the real function output (with mocked Supabase and Gemini) against the same types and validator the frontend uses. 3. Map API errors to the correct UI states (AI fallback, database failure, empty). 4. Remove the fixture from the production path but keep it for tests. NON-GOALS: No new features or redesign. SKILLS: context-engineering, api-and-interface-design. FILE OWNERSHIP: RS-00 may edit src/lib/api.ts and add one contract test under tests/api/. If any other file needs changing, write a request in docs/SPRINT_STATUS.md. TESTS: run the full suite. ACCEPTANCE: App shows real destinations locally (explain how to run it). Contract test passes. EXPECTED OUTPUT: Test summary and the exact way I can see it work. [FOOTER]
```

P05-A — Browser journeys (RS-23, thread RS-03)

```
OBJECTIVE: Write and run browser tests for the user journeys. CONTEXT: Playwright is installed only if P00-A said so. If it is not, ask me before installing @playwright/test and browsers (large download). If I decline, write a manual test script in docs/TEST_RESULTS.md instead. SCOPE: Use Playwright network interception to mock /api/discover for deterministic runs. Journeys: (1) structured-only happy path; (2) free-text path with applied banner; (3) missing selection shows the message; (4) Like then Unlike; (5) Dislike replaces a card and keeps other slots; (6) pool exhaustion; (7) new search resets feedback; (8) free text not applied shows the amber notice; (9) images blocked shows the fallback; (10) 375x812 viewport has no horizontal scroll. NON-GOALS: Do not edit src/** or netlify/**. Report bugs with exact steps to reproduce. SKILLS: test-driven-development (browser testing). FILE OWNERSHIP: tests/e2e/**, docs/TEST_RESULTS.md. TESTS: run all journeys. ACCEPTANCE: Honest results table with pass, fail, not-run. EXPECTED OUTPUT: Results table and a bug list with owner thread for each. [FOOTER]
```

P05-B — Security review (RS-24, thread RS-03)

```
OBJECTIVE: Check for exposed secrets and weak access rules. SCOPE: 1. Search the repository, git history and the built dist folder for the values in .env WITHOUT printing them (compare programmatically and report only match or no match). 2. Confirm .env is git-ignored and was never committed. 3. Run scripts/check-rls.ts (I will confirm the migration has been applied) and report. 4. Confirm the browser bundle contains no Supabase or Gemini keys and makes no direct calls to Supabase or Gemini. 5. Confirm server-side validation exists for every request field and that raw free text is never logged. 6. Run "npm audit" and summarize only. Do not run "npm audit fix" or use --force. 7. If a security-related skill is installed, list it and use it; otherwise follow this checklist. NON-GOALS: Do not change code. Report findings with severity and the owner thread. FILE OWNERSHIP: docs/TEST_RESULTS.md (append). SKILLS: none verified. TESTS: those above. ACCEPTANCE: A findings table. "No issue found" is allowed only for checks you actually ran. EXPECTED OUTPUT: Findings table. [FOOTER]
```

P05-C — Failure drills (RS-25, thread RS-03)

```
OBJECTIVE: Prove failure behavior matches the PRD. SCOPE: Using mocks or environment overrides (never my real key in output) test: Gemini timeout (set GEMINI_TIMEOUT_MS very low), missing GEMINI_API_KEY, missing GEMINI_MODEL, invalid JSON from Gemini, Gemini HTTP error, Supabase failure. For each, record the API response and what the UI shows. EXPECTED BEHAVIOR (PRD FR-5, section 13): Gemini problems return structured results with free_text_applied=false and the amber notice. Database problems show a separate error state. Nothing crashes. NON-GOALS: No code changes. Report mismatches with the owner thread. FILE OWNERSHIP: tests/e2e/**, tests/api/** (add tests only), docs/TEST_RESULTS.md. SKILLS: test-driven-development. TESTS: as above. ACCEPTANCE: A table: drill, expected, actual, pass or fail. EXPECTED OUTPUT: The table. [FOOTER]
```

P06-A — Production deploy and smoke test (RS-27, thread RS-00)

```
OBJECTIVE: Prepare and verify the production release. SCOPE: 1. Check netlify.toml and that docs/DEPLOYMENT.md lists all environment variable NAMES (no values). 2. Create scripts/smoke.ts that takes a base URL and checks: home page loads; /api/health ok; /api/discover with a valid structured request returns the contract shape and every candidate satisfies the budget and month rules; an invalid request returns a 400; a request with free text returns a valid response (free_text_applied true or false, both acceptable, report which). Also run the valid request 10 times and report minimum, median and maximum time, without promising targets. 3. Search the deployed files for the first 8 characters of my GEMINI_API_KEY without printing them and report match or no match. 4. Update docs/DEPLOYMENT.md with the URL and smoke results. NON-GOALS: Do not deploy by yourself. Tell me which button to press in Netlify, then wait. FILE OWNERSHIP: scripts/smoke.ts, docs/DEPLOYMENT.md, docs/TEST_RESULTS.md. SKILLS: none verified. TESTS: run smoke against the live URL after I deploy. ACCEPTANCE: Smoke output pasted honestly. EXPECTED OUTPUT: Results table and any failing check. [FOOTER]
```

P07-A — README and evidence (RS-29, thread RS-00)

```
OBJECTIVE: Write README.md and an evidence list. No claims beyond what tests and the live site prove. SCOPE: README sections: what RoamSense is; what it is not (curated demo dataset, estimated costs, not live prices or safety info); how the AI is used and where it is not; stack; setup (environment variable names only); how to run tests; live URL; known limits; P1 items not built. Evidence list: file names for test output, smoke output and the screenshots I will add. NON-GOALS: No marketing language. Do not say "verified" about data. Do not claim latency or tester results unless they are recorded in docs/TEST_RESULTS.md. FILE OWNERSHIP: README.md, docs/**. SKILLS: none. TESTS: none. ACCEPTANCE: Every claim traces to PRD or recorded results. EXPECTED OUTPUT: The README and the evidence list. [FOOTER]
```

P-FIX — Bug fix template (any thread that owns the file)

```
A bug: [WHAT I DID], [WHAT I EXPECTED], [WHAT HAPPENED, WITH EXACT ERROR TEXT OR SCREENSHOT DESCRIPTION]. First reproduce it with a failing test. Explain the cause in one plain line. Then fix it with the smallest change inside the files this thread owns. If the cause is in a file this thread does not own, stop and write a request in docs/SPRINT_STATUS.md. Re-run the full test suite and report real results. [FOOTER]
```

P-HANDOFF — Before leaving a thread

```
Prepare a handoff. Update docs/SPRINT_STATUS.md. Then print: current phase; tasks completed with the commit or tag; files changed; latest test command and result; open blockers; requests for other threads; next action; and anything the next thread must not change.
```

14. Testing and Quality Gates — Companion Completion

This section completes the supplied playbook; it does not replace PRD v6.1.

Gate	Required evidence	Do not advance if
G0 Spec	PRD accessible, decisions recorded, test matrix prepared	PRD missing or contract conflicts unresolved
G1 Foundation	Local build and unit tests pass; starter site and health endpoint publicly reachable	Broken build or no public skeleton
G-D Seed	Validated CSV, six-region coverage, documented estimates and image fallbacks	Invalid schema or fabricated verification
G2 Backend	Real database query, API contract tests, deterministic ranking tests, AI parser/fallback tests	Hard filters or RLS fail
G3 Frontend	Form, cards, Like/Unlike/Dislike, empty/error states and responsive layout tested	Feedback logic or accessibility-critical controls broken
G4 Integration	Browser journey uses real API, not fixture; no contract mismatch	Demo-only data accidentally remains enabled
G5 QA	Regression tests, security review and failure drills pass	Secrets exposed or P0 test failing
G6 Production	Public URL, working discovery, AI parser, fallback and mobile smoke tests	Only local app works

> **Test evidence format:** date/time; environment; exact command or browser action; expected result; observed result; PASS/FAIL/NOT RUN; follow-up owner. Never substitute an AI assertion for executed evidence.

15. Deployment and Release Playbook — Companion Completion

1. Build and test locally; verify `.env` is ignored and no secrets appear in committed files or browser bundle.
2. Push the reviewed project to a GitHub repository through Codex/GitHub UI after approval.
3. Import the GitHub repository in Netlify. Confirm `npm run build`, `dist`, `netlify/functions`, and redirects match `netlify.toml`.
4. Configure **server-side** environment variables in Netlify: `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `GEMINI_TIMEOUT_MS`. Never paste their values into chat.
5. Redeploy after changing environment variables; open `/api/health` and submit a real request to `/api/discover`.
6. Run the P06-A smoke tests against the actual public URL, including a free-text request that demonstrably applies AI tags; record any fallback.
7. Test a full discovery and feedback journey on a mobile device. Record the public URL and test outcomes in `docs/DEPLOYMENT.md`.

Important: *Netlify dashboard labels and limits may change. Verify current behavior in your account rather than assuming this playbook's descriptions are authoritative.*

16. Progress Tracking and Handoff Templates — Companion Completion

`docs/SPRINT_STATUS.md` template:

```
# Sprint Status Product: RoamSense Phase: 0 — Specification freeze Completed: None Current blocker: None Latest tests: Not run Next action: RS-01 — Place PRD in project docs folder Deployment: Not deployed Remaining critical work: Foundation, data, backend, frontend, integration, QA, release Requests: None
```

`docs/TEST_RESULTS.md`: append rows with `Date | Gate | Test | Expected | Actual | Status | Evidence/Notes`.

Thread handoff: Use P-HANDOFF from Section 13; begin next thread with P-RESUME. File-based status is handoff source, not assumed cross-chat memory.

17. Troubleshooting and Recovery Guide — Companion Completion

Symptom	First check	Next safe action
`npm` or build fails	Copy exact error and failing command	Use P-FIX in owning thread; do not reinstall everything
Missing environment variable	Compare variable **names** in `.env.example` / Netlify dashboard	Add it through the appropriate UI, then restart/redeploy
Gemini fails	Confirm selected model and actual quota; inspect sanitized server error	Preserve structured-only fallback; test live AI again before claiming completion
Supabase returns no rows	Check seed import, month spelling, tier, and RLS	Run a narrow read-only query; don't disable RLS
Like/Dislike behaves oddly	Reproduce with small fixture	Add failing regression test before code change
Netlify returns 404 for `/api/discover`	Inspect `netlify.toml`, function filename, deployment logs	Fix route and redeploy; do not change product architecture
Site works locally but not live	Inspect Netlify deploy logs and production env names	Re-run smoke tests after correction
Skill/plugin missing	Check P00-A inventory	Continue with ordinary Codex and explicit test requirements

> **Recovery rule:** Inspect `git status` and recent commits before reverting. Avoid `git reset --hard`, force pushes or deleting unknown files without explicit review.

18. Final Submission and Demonstration Checklist — Companion Completion

- [ ] RoamSense live URL opens publicly.
- [ ] Natural-language input demonstrably changes `want/avoid` interpretation when Gemini is available.
- [ ] Budget and month filters never silently relax.
- [ ] Like/Unlike/Dislike updates the replacement ranking.
- [ ] Actual test outcomes recorded; known limitations disclosed.
- [ ] Mobile walkthrough and screenshots captured.
- [ ] Short demo video recorded from the live site.
- [ ] PPT shows problem, user, solution, AI role, architecture, live demo, evidence, limitations and next steps.
- [ ] Wedding and Feedback tracks each have their own approved PRD, implementation, public URL, video and PPT as required by the assignment.
- [ ] Submission requirements and actual deadlines confirmed with the assignment instructions.

19. Interactive Guided Execution Protocol — Companion Completion

When the user says **Start the sprint**, give **one action at a time** in this format:

> **Product / Phase / Step ID:** …
> **Goal:** …
> **Where:** …
> **Exact action or prompt:** …
> **Expected result:** …
> **Verification:** …
> **Reply:** A — complete / B — error / C — cannot locate option.

Do not advance before confirmation. For errors, request only specific error text/screenshot and propose one corrective action. Track progress through `docs/SPRINT_STATUS.md`.

20. Quick Start — My First Action — Companion Completion

RS-01: Put the authoritative PRD in your existing RoamSense project.

Your Codex project was previously created on the **W: drive**. **Do not create a second `C:\Projects\roamsense` folder** merely because Section 10 uses that example path. Open the existing RoamSense project folder, create `docs` if needed, and copy `RoamSense_PRD_v6.1_Final.md` to `docs/PRD_v6.1.md`. Verify the file begins with `# RoamSense — Product Requirements Document (PRD) v6.1`.

Reply A when present, B if you encounter an error, or C if you cannot find the project folder.

Editor's Notes — Cross-Section Corrections

1. **Project location:** Section 10's `C:\Projects\roamsense` is illustrative and conflicts with the existing W: drive Codex project. Use the existing project location.
2. **Section completeness:** The uploaded `p4a.md` stops at Section 13. Sections 14–20 above complete the guide; they are concise operational supplements, not additional Claude-authored sections.
3. **Gate labels:** Section 10's RS-04 completion line says SO-1 through SO-9; P00-B requires SO-1 through SO-11. Follow P00-B (eleven decisions).
4. **Placeholder footer:** `[FOOTER]` in Section 13 is a placeholder. When copying a prompt to Codex, replace it with the actual footer text printed at the start of Section 13. Do not paste the literal placeholder alone.
5. **Ownership and status:** Every thread may update `docs/SPRINT_STATUS.md` for handoffs despite `docs/**` generally belonging to Lead. Treat that status file as a deliberate shared exception.
6. **Seed data:** CSV draft values and source notes are not factual verification. The schema's required `image_url` is compatible with an empty string for a fallback image, but the UI must not pretend a blank URL is a working image.
7. **Approval:** The guide's 'commit and tag after every gate' is optional housekeeping, not a reason to delay functional delivery or run destructive Git commands.